

DEEP LEARNING

Assignment 4

Domain-Specific Transformer for Stack Overflow Q&A

Saad Ali - MSDS25066

1. Problem Statement:

Imagine you're a Python developer stuck on a problem at 2 AM — no one to ask, deadline creeping closer. You type your question into Stack Overflow and just *hope* someone answers. Now, what if there was something that could answer you right away, every time?

That's exactly the heart of this assignment.

We're not borrowing some giant, pre-built AI and slapping a new label on it. We're rolling up our sleeves and building a language model **from scratch** — brick by brick, line by line. And this model has one job, one purpose, one passion — understanding Python questions and actually giving back answers that make sense.

The way it learns is beautifully simple. You hand it a question — the kind real developers ask every single day on Stack Overflow — complete with a title and a description of the problem. The model reads it, processes it, and teaches itself over time how to respond like someone who genuinely *gets* Python.

It's not trying to be ChatGPT. It's not trying to know everything about everything. It's a specialist. A dedicated little assistant that has spent its entire existence living and breathing Python problems — and honestly? That's what makes it special.

So how does the model actually *think*? How does it learn to string words together in a way that feels like a real answer?

It all comes down to a beautifully simple idea — **predict what comes next**.

Imagine reading a sentence one word at a time, and before each new word appears, you take a guess at what it's going to be. You're not guessing randomly — you're using everything you've already read to make an educated guess. That's exactly how this model learns. It reads through thousands of real Stack Overflow answers, and at every single step, it asks itself — *"given everything I've seen so far, what word is most likely coming next?"* Over time, by getting better and better at that one humble task, it quietly teaches itself the rhythm, the logic, and the structure of how Python answers are actually written.

This style of learning has a formal name — **causal language modeling** — but don't let the fancy term intimidate you. It's just the model learning the way humans naturally do — one step at a time, always moving forward, never peeking ahead.

And the engine powering all of this? A **decoder-only Transformer** — the same GPT-style architecture that changed the AI world forever. It traces its roots back to a now-legendary research paper called *"Attention is All You Need"* published by Vaswani and his team back in 2017. That paper didn't just introduce a new technique — it rewrote the rules of what AI could do with language. And here we are, building something inspired by that very same breakthrough.

The complete pipeline covers:

- Data loading, cleaning, and preprocessing of raw Stack Overflow CSV files
- Custom Transformer model architecture built from scratch in PyTorch
- Training with AdamW optimizer, CosineAnnealingLR scheduler, and early stopping
- Evaluation using Cross-Entropy Loss, Perplexity, and BLEU score
- Two experiments with different hyperparameters for comparison
- Interactive Q&A inference using top-k sampling with temperature control

2. Dataset Description:

Every great model needs great data — and this one is no exception

The raw material powering this entire project comes from one of the most trusted sources in the developer world — **Stack Overflow**. Not a curated textbook, not an artificial dataset cooked up in a lab — but *real* questions asked by *real* developers who were genuinely stuck, and *real* answers written by *real* people who took the time to help.

The data arrives in three files — **Questions.csv**, **Answers.csv**, and **Tags.csv** — all pulled straight from Stack Overflow's public data dump, and all encoded in **latin-1** format, just as the assignment specifies. Nothing fancy, nothing exotic — just honest, raw data waiting to be shaped into something meaningful.

And the scale of it? Honestly, it's humbling.

Over **607,000 questions** — each one representing a moment where someone sat at their keyboard, frustrated, curious, or just plain lost. Over **987,000 answers** — each one a small act of generosity from someone in the community who said "*I know this, let me help.*"

But we don't throw everything at the model at once. Instead, we do something smart — for every question, we find its **single best answer**, the one the community voted up the most. That's the answer worth learning from. That's the signal worth chasing.

From that massive ocean of knowledge, we carefully scoop out **3,000 question-answer pairs** — enough to train on, enough to learn from, without drowning the process. Those 3,000 pairs are then split with intention — **80% for training**, where the real learning happens, **10% for validation**, where we keep the model honest, and **10% for testing**, where we find out if any of it actually worked.

Linking Strategy:

Normally, Stack Overflow data comes with a convenient shortcut — an **AcceptedAnswerId** column that tells you exactly which answer the person who asked the question officially blessed as "*the one.*" Clean, simple, done.

Except — this version of the dataset didn't have that column.

So instead of giving up or settling for something messy, we figured out a smarter way to get to the same destination. For every question in the dataset, we looked at all the answers the community had written — and simply picked the one that earned the **highest score**. Because when thousands of developers collectively upvoted an answer, that's not random. That's the community quietly agreeing — *"yes, this one actually helps."* It's not a perfect substitute, but it's a deeply honest one.

After carefully matching questions to their best answers this way, we ended up with a rich pool of **539,238 valid question-answer pairs** — over half a million real conversations between confused developers and helpful ones. From that pool, we hand-picked **3,000 pairs** using stratified random sampling, locking in a **random seed of 42** so that anyone, anywhere, running this same code gets the exact same result. Reproducibility isn't just a technical detail — it's a form of scientific respect.

Tags.csv was explored during initial data analysis but not used in the final pipeline, as the Q&A generation task only requires question text and answer text.

Cleaning the Data — Making It Fit for a Model to Learn From:

Raw Stack Overflow data is wonderful — but it's also a little wild. Posts are wrapped in HTML tags, riddled with inconsistent capitalization, and scattered with special characters that mean something to a browser but nothing to a language model. Before the model sees a single word, the data goes through a careful, four-step cleaning pipeline.

- **Strip Away the HTML** Stack Overflow answers are written for web pages — so they come loaded with tags like `<p>`, `<code>`, and `<br>`. Using **BeautifulSoup**, we peel all of that away and extract just the clean, readable text underneath. What the model gets is what a human would actually *read* — not what a browser would *render*.
- **Make Everything Lowercase** To the model, "Python" and "python" shouldn't be two different things — they're the same word, just dressed differently. So we bring everything down to lowercase, creating a level playing field where the meaning of a word matters more than how someone happened to capitalize it that day.
- **Remove the Noise** Using **Regex**, we quietly sweep away anything that doesn't add meaning — stray symbols, unusual characters, digital clutter. What survives are the things that actually matter — letters, numbers, and the basic punctuation marks that carry real meaning: periods, commas, question marks, exclamation points.
- **Breathe Out the Extra Space** Multiple spaces, stray gaps, awkward indentations — they all get collapsed into a single, clean space. The text comes out neat, consistent, and ready.

Giving the Model a Language of Its Own:

Once the data is clean, each question-answer pair gets stitched together into one continuous training sequence — using a format the model will come to recognize as its native tongue:

```
<SaadAli_MSDS25066> question_title . question_body [SEP] answer_body  
</SaadAli_MSDS25066>
```

Every sequence opens with a custom token — **<SaadAI_MSDS25066>** — a unique signature that marks the beginning of a learning example. The question's title and body come first, followed by a **[SEP]** token that acts like a quiet dividing wall — saying *"everything before me is the question, everything after me is the answer."* The sequence then closes with the matching end token, wrapping the whole thing up like a complete thought.

And during inference — when the model is actually put to the test — it's handed only the question side, right up to that **[SEP]** marker. From there, it's on its own. It has to generate the answer, word by word, all by itself — just like a developer would.

Tokenization:

Property	Value
Tokenizer	openai-community/openai-gpt (GPT-1)
Vocabulary Size	40,479 tokens (after adding [PAD])
Pad Token	[PAD] (token ID: 40,478)
Max Length	128 tokens (pad/truncate)
Return Type	PyTorch tensors

3. Model Architecture — MiniGPT:

At its core, this model is a **decoder-only Transformer**. You might also hear it called a causal language model, or a GPT-style model. Different names, same beautiful idea. But to really appreciate what that means, it helps to take one small step back.

When the original Transformer was introduced to the world in that landmark 2017 paper — "*Attention is All You Need*" — it came with two distinct halves working in harmony. An **encoder** on one side, reading and deeply understanding the input. A **decoder** on the other side, taking that understanding and producing an output. Together, they were built for tasks like translation — where you read a sentence in French and write it back in English. Two languages, two roles, two sides of the same brain. But here's the thing — we're not translating anything.

We're not taking one sequence and converting it into another. We're doing something more organic, more human. We're **generating**. We're looking at a question and *continuing* the conversation — one word at a time, one thought at a time, always moving forward, never looking back. And for that kind of task? You don't need both halves. You only need the **decoder**.

So that's exactly what we use — just the decoder stack, stripped down, focused, and purpose-built for generation. There's no encoder sitting on the side feeding it a compressed understanding of the input. Instead, the decoder does everything itself — it reads the question, holds it in memory, and generates the answer autoregressively, which is just a fancy way of saying it writes the answer **one token at a time**, using everything it has already written to decide what comes next.

It's honestly a lot like how a human expert would answer a question. You don't process the question in one part of your brain and answer with another. You read it, you understand it, and you just — *respond*. Naturally. Sequentially. One word flowing into the next. That's the decoder-only Transformer. That's the GPT-style model. And that's the engine at the heart of everything this project is trying to do.

Architecture Overview:

Layer	Component	Description
1	Token Embedding	<code>nn.Embedding(40479, d_model)</code> — maps each token ID to a dense vector
2	Positional Encoding	Sine/cosine fixed encoding — adds position information to embeddings
3	Transformer Block × N	N stacked blocks
4	Final LayerNorm	Normalizes output activations before projection
5	Output Projection	<code>nn.Linear(d_model, 40479)</code> — maps vectors to vocabulary logits

Transformer Block (Internal Structure):

If the Transformer is the brain, then the **Transformer Block** is the neuron — the fundamental unit where actual thinking takes place. And inside every single one of these blocks, there's a carefully designed rhythm, a two-step dance that happens over and over again, each time making the model's understanding just a little bit richer, a little bit deeper.

Each Transformer Block contains **two sub-layers** — two distinct moments of processing, two opportunities to learn something new. But what makes these sub-layers truly special isn't just what they do — it's *how* they're protected.

Sub-layer	Component	Formula
1a	Masked Multi-Head Self-Attention	$Attention(Q, K, V) = softmax(QK^T / \sqrt{dk}) \times V$
1b	Add & LayerNorm	$x = LayerNorm(x + dropout(attention_output))$
2a	Feed Forward Network	$FFN(x) = Linear(ReLU(Linear(x)))$ [$d_model \rightarrow d_ff \rightarrow d_model$]
2b	Add & LayerNorm	$x = LayerNorm(x + dropout(ffn_output))$

Attention Mechanism:

If there's one idea that sits at the absolute heart of the Transformer — one mechanism that makes everything else possible — it's **attention**. Not attention in a vague, metaphorical sense. Actual, mathematical, deliberately calculated attention. The model's way of asking, at every single step — *"out of everything I've seen so far, what should I be focusing on right now?"*

To understand how attention works, imagine you're searching for a book in a vast library.

You walk in with a question in your mind — *"I'm looking for something about recursion in Python."* That question in your head? That's your Query — what you're looking for, what you need right now. Every book on the shelf has a label on its spine — a title, a topic, a brief description. Those labels? Those are the Keys — what each piece of information *says it's about*, the signal it puts out into the world. And the actual content inside each book — the pages, the explanations, the knowledge? That's the **Value** — the real substance that gets delivered once you've decided something is worth your attention.

Attention works by comparing your Query against every Key in the room — figuring out which ones match most closely — and then pulling back a weighted blend of the corresponding Values. Not just the single best match, but a rich mixture of everything that felt relevant, blended together in proportion to how well each one matched your question.

The Formula — Beautiful in Its Simplicity:

$$\text{Attention}(Q, K, V) = \text{softmax}(Q @ K^T / \sqrt{d_k}) @ V$$

Where Q (Query), K (Key), and V (Value) are linear projections of the input. The scaling factor $\sqrt{d_k}$ prevents vanishing gradients when d_k is large

A single attention operation is powerful — but it can only focus on *one kind of relationship at a time*. And language is far too rich, far too layered for that. A single sentence can carry grammatical relationships, semantic connections, contextual dependencies, and subtle long-range references — all at once, all intertwined. So instead of running attention just once, we run it **multiple times in parallel** — each time through a different lens, each time looking for something different. These are the **attention heads**.

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h) @ W_o$$

$$\text{where } \text{head}_i = \text{Attention}(Q @ W_{q_i}, K @ W_{k_i}, V @ W_{v_i})$$

Causal Mask:

The problem exists in language model training — and it's a surprisingly sneaky one. During training, the model sees the *entire* sequence at once — both the question *and* the answer, laid out together in full. Without any guardrails, nothing stops it from peeking ahead — from letting future words quietly influence how it processes the current one. And if that happens, the model learns nothing real. It just learns to cheat. So we give it a **blindfold**. A mathematically precise, beautifully enforced blindfold.

This blindfold has a formal name — the **causal mask**, or more visually, the **lower-triangular mask**. And its logic is almost poetically simple.

$$\text{mask}[i][j] = 1 \text{ if } j \leq i \text{ else } 0$$

The first token can only see itself. The second token can see the first and itself. The third can see the first, the second, and itself. And so on — each position living only in its own present moment and past, never borrowing knowledge from the future it hasn't earned yet.

But a mask is only useful if it actually *works*. So how do we make sure the model genuinely can't peek ahead? We make peeking ahead **mathematically catastrophic**.

$$\text{scores} = \text{scores.masked_fill}(\text{mask} == 0, -1\text{e}9)$$

Every attention score that corresponds to a future token — every cell in the grid where $j > i$ — gets filled with a number so deeply, absurdly negative that it might as well be negative infinity. -1,000,000,000.

Then, when softmax runs over these scores to produce attention weights, those future positions don't just get low weights — they get weights so vanishingly small they effectively become **zero**. They contribute nothing. They influence nothing. They might as well not exist. The future has been walled off. Not by asking the model nicely to ignore it — but by making it *mathematically impossible* to pay attention to it.

Because of this mask, every prediction the model makes during training is **genuine**. When it tries to guess the next word, it does so with only the words that came before it — exactly the same information it will have during real inference, in the real world, when there *is* no answer to peek at. The training conditions and the real-world conditions become identical. There's no gap between how the model learned and how it performs.

Positional Encoding — Giving the Model a Sense of Place:

Unlike a human reader — or even older neural network designs that processed words one at a time — the Transformer looks at the **entire sequence all at once**. Every token, simultaneously, in one sweeping glance. Which is exactly what makes it so powerful. But it also creates a quiet, dangerous blindspot. Without any additional help, the Transformer has absolutely no idea where anything is. To it, a sentence and its scrambled version look identical — just a bag of tokens floating in space, with no beginning, no end, no sense of what came first or what came last. Word order — the very thing that gives language its meaning — becomes completely invisible

So, we have to give the model something it's missing. We have to teach it to feel **position**.

The fix is something called **Positional Encoding** — and the approach borrowed from the original "*Attention is All You Need*" paper is as elegant as it is ingenious. Instead of learning position from data — which would take time and eat up precious parameters — we simply *calculate* it, Mathematically.

$$\text{PE}(\text{pos}, 2i) = \sin(\text{pos} / 10000^{(2i / d_{\text{model}})})$$

$$\text{PE}(\text{pos}, 2i + 1) = \cos(\text{pos} / 10000^{(2i / d_{\text{model}})})$$

Every position in the sequence — position 1, position 2, position 47, position 312 — gets assigned its own unique **fingerprint**. A fingerprint made not of ink, but of **waves**. Sine waves and cosine waves, oscillating at carefully chosen frequencies, each one tuned to a different dimension of the model's internal representation. Some waves oscillate quickly — completing many full cycles across the length of the sequence. They're good at distinguishing positions that are *close together*. Others oscillate slowly — barely completing one full cycle even across a very long sequence. They're good at capturing *broad, long-range position information*. Together, these waves at different frequencies create a rich, multi-dimensional signal — one that is **unique for every position**, and yet **smoothly consistent** across the whole sequence. No two positions produce exactly the same fingerprint. And positions that are close to each other produce fingerprints that are close to each other — which means the model can naturally sense not just *where* something is, but *how far* it is from everything else.

The choice to pair **sine** for even dimensions and **cosine** for odd dimensions isn't arbitrary either. Sine and cosine are natural companions — shifted versions of each other, always in conversation, always completing what the other started. Together, they ensure that at every frequency, position is encoded from *two complementary angles* — giving the model a richer, more robust sense of location than either wave could provide alone.

And here's the final, quietly beautiful detail — these positional encodings aren't attached to the token embeddings as a separate tag or label. They're **added directly into them**. The meaning of a word and its position in the sequence are fused together into a single unified vector — inseparable, intertwined, two aspects of the same signal. So, when the model processes any token, it isn't looking at *just* what the word means, or *just* where it sits in the sequence. It's looking at **both at the same time** — a single, rich representation that carries the full picture.

Model Configuration (Both Experiments):

Parameter	Value	Explanation
vocab_size	40,479	Size of tokenizer vocabulary
d_model	128	Embedding and hidden state dimension
num_heads	4	Number of parallel attention heads (d_k = 32 each)
num_layers	3	Number of stacked Transformer blocks
d_ff	512	Feed-forward hidden dimension (4 × d_model)
max_seq_len	128	Maximum input sequence length Maximum input sequence length
dropout	0.1	Dropout rate for regularization
Total Params	10,997,023	~11 million learnable parameters

4. Training Setup:

Loss Function:

Every time the model predicts the next token, it doesn't just pick one word — it produces a **spread of confidence** across every word in the vocabulary. Maybe 60% confident it's *"list"*, 20% for *"tuple"*, and so on. Cross-entropy then looks at the *actual* correct word and measures the gap — how far the model's confidence was from where it should have been. High loss means confused. Low loss means learning. Every training step tries to close that gap just a little more.

loss

```
= CrossEntropyLoss(logits.reshape(-1, vocab_size), targets.reshape(-1), ignore_index  
= 40478)
```

Sequences in a batch are different lengths, so shorter ones get padded with a special filler token — **ID 40478**. These tokens carry no real meaning, so we tell the loss function to simply ignore them. The model gets judged only on real words. Real knowledge. Nothing else.

At every training step, the loss function points the model in one direction — *"be more confident in the right answer, less confident in everything else."* Follow that signal long enough, consistently enough, and the model finds its way.

Optimizer — AdamW:

Once the model knows how wrong it is, something has to actually *fix* it. That's AdamW — the standard optimizer for Transformer training, and for good reason.

Regular **Adam** was already a leap forward — giving each parameter its own personalized learning rate, adapting intelligently at every step. But it handled weight decay messily, tangling it into the gradient update where it didn't belong. **AdamW** simply cleans that up — decoupling weight decay and applying it independently, the way it was always meant to work.

Without any restraint, models overfit — they memorize the training data so deeply that they completely fall apart on anything new. Weight decay acts as **L2 regularization**, gently penalizing weights that grow too large, constantly nudging the model toward solutions that are not just correct — but *generally* correct. Solutions that hold up in the real world, not just on the training set. AdamW learns fast, adapts smartly, and stays honest — making it the quiet backbone of reliable Transformer training. We take weight decay as 0.01.

Learning Rate Scheduler — CosineAnnealingLR:

At the start of training, the model knows nothing — so we press it hard. Big steps, fast progress, aggressive learning. But as the model matures and gets closer to a good solution, those same big steps start doing more harm than good. They overshoot. They undo progress.

So we ease off — **gradually, smoothly** — following a cosine curve all the way down to a gentle minimum of **1e-6**. No sudden drops. No jarring transitions. Just a natural, steady slowdown that gives the model exactly what it needs — **speed early, precision late**.

Early Stopping & Checkpointing:

More training doesn't always mean better results. Sometimes the model quietly peaks — and every step after that just makes things worse.

Early stopping catches that moment. If the validation loss fails to improve for **5 consecutive epochs**, training simply stops. No point pushing further. The model has already given its best — continuing would only teach it to memorize, not generalize. And the best version of the model never gets lost. **Checkpointing** saves it automatically — every time the validation loss hits a new low, the model's weights get saved to **weights/best_model.pt**, quietly preserving that peak moment for later. When final testing comes around, it's not the last version of the model that gets evaluated — it's the **best** one. The one that performed most honestly, before overfitting ever had a chance to creep in.

Evaluation Metrics:

Training a model is one thing. Knowing whether it genuinely learned something is another. Three metrics do that job here — each one measuring quality from a slightly different angle.

- **Cross-Entropy Loss — The Training Signal:** The same loss used during training becomes the first evaluation metric. Simple and direct — it measures how confidently the model predicted the right token at every position. Lower is better. **Zero would mean perfect prediction** — which never happens in practice, but gives us something to chase.
- **Perplexity — The Intuition Check:** Perplexity is just the loss made human-readable.

$$PPL = \exp(loss)$$

It answers one honest question — *"at each step, how many words was the model genuinely torn between?"* A **perplexity of 10 means** the model felt like it had **10 equally reasonable choices**. A perplexity of 2 means it was nearly certain. **Lower is better** — and a steadily falling perplexity is one of the most satisfying signs that real learning is happening.

- **BLEU Score — The Reality Check:** BLEU compares the model's generated answer against the real answer — measuring how much n-gram overlap exists between the two. It runs from 0 to 1. Zero means nothing matched. One means perfection. For open-ended generation like this — where many valid answers exist — anything **above 0.3** is considered genuinely decent. It's an imperfect metric, but an honest one.

Three metrics. Three perspectives. One shared goal — a model that doesn't just train well, but actually *works*.

5. Experiment 1 — Baseline (Learning Rate = **3e-4**)

Hyperparameters:

Hyperparameter	Value
learning_rate	3e-4 (0.0003)
batch_size	16
epochs	15
d_model	128
num_heads	4
num_layers	3
d_ff	512
dropout	0.1
optimizer	AdamW (weight_decay=0.01)
scheduler	CosineAnnealingLR (T_max=15, eta_min=1e-6)

Epoch-by-Epoch Training Results:

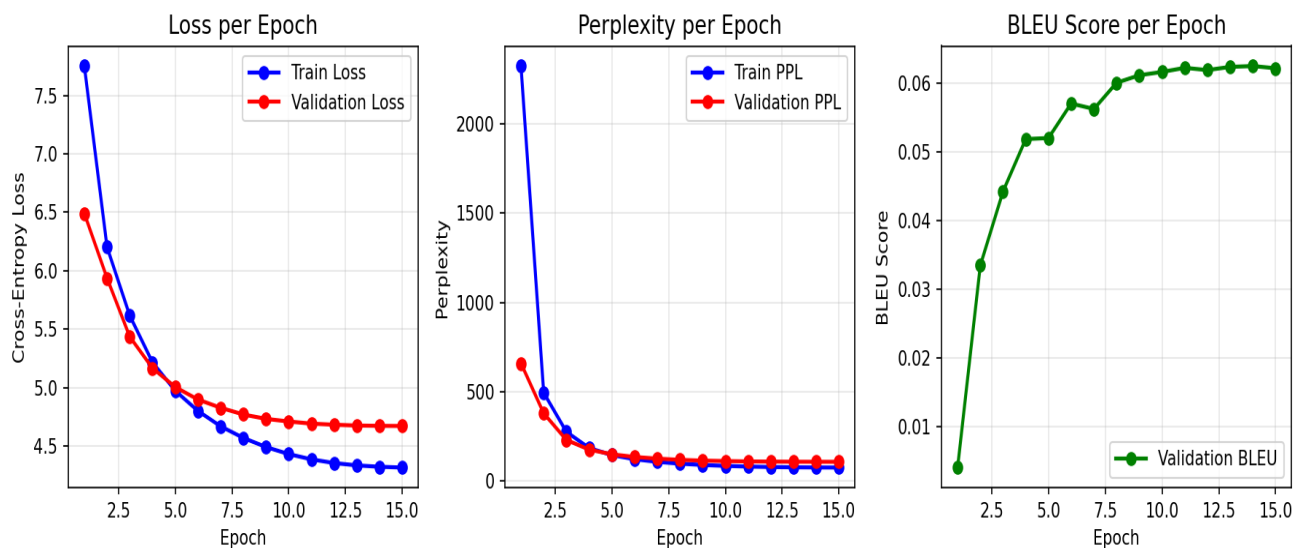
Experiment 1 — Full Training Log

Epoch	Train Loss	Train PPL	Val Loss	Val PPL	Val BLEU	LR
1	7.7506	2322.98	6.4907	658.99	0.0041	000297
2	6.2024	493.93	5.9372	378.86	0.0335	0.000287
3	5.6168	275.00	5.4363	229.60	0.0442	0.000271
4	5.2178	184.53	5.1682	175.59	0.0518	0.000251
5	4.9753	144.79	5.0068	149.42	0.0520	0.000225
6	4.8025	121.81	4.9001	134.30	0.0570	0.000197
7	4.6711	106.82	4.8291	125.10	0.0562	0.000166
8	4.5715	96.69	4.7719	118.15	0.0600	0.000135
9	4.4937	89.45	4.7345	113.81	0.0611	0.000104
10	4.4335	84.23	4.7122	111.30	0.0616	0.000076

11	4.3885	80.50	4.6939	109.28	0.0622	0.000050
12	4.3568	78.00	4.6845	108.26	0.0619	0.000030
13	4.3367	16.46	4.6777	107.52	0.0623	0.000014
14	4.3250	75.56	4.6749	107.22	0.0624	0.000004
15	4.3184	75.07	4.6739	107.11	0.0621	0.000001

Final Results

Metric	Training	Validation	Test
Loss	4.3184	4.6739	4.6760
Perplexity	75.07	107.11	107.34
BLEU Score	-----	0.0624	0.0618



Observations:

- Loss decreased consistently every single epoch — no early stopping was triggered
- Validation perplexity dropped from 659 (epoch 1) to 107 (epoch 15), a 6x improvement
- BLEU score improved steadily from 0.004 → 0.062, indicating the model learned meaningful answer patterns
- The small gap between train PPL (75) and val PPL (107) indicates mild, acceptable overfitting
- CosineAnnealingLR gradually reduced LR from $3e-4$ to $\sim 1e-6$, providing smooth convergence

6. Experiment 2 — Higher Learning Rate (Learning Rate = 1e-3):

The only change from Experiment 1 is the learning rate, increased from 3e-4 to 1e-3 (3x higher). All other hyperparameters remain identical.

Epoch-by-Epoch Training Results:

Experiment 2 — Full Training Log (stopped at epoch 13)

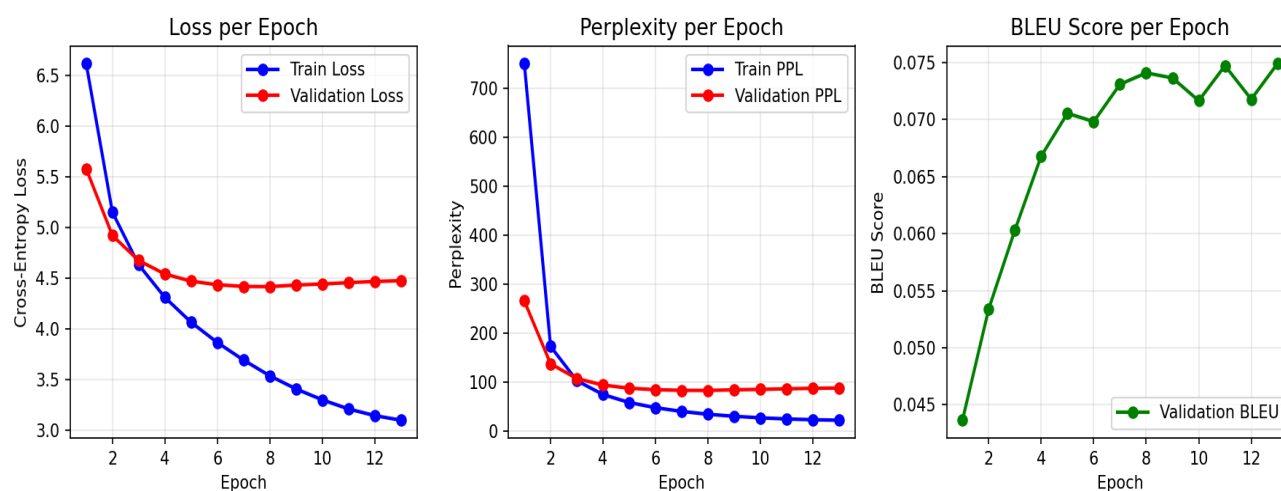
Epoch	Train Loss	Train PPL	Val Loss	Val PPL	Val BLEU	LR
1	6.6196	749.64	5.5820	256.60	0.0436	0.000989
2	5.1523	172.83	4.9224	137.33	0.0534	0.000957
3	4.6337	102.90	4.6757	107.30	0.0603	0.000905
4	4.3109	74.51	4.5424	93.92	0.0668	0.000835
5	4.0657	58.31	4.4722	87.55	0.0705	0.000750
6	3.8655	47.73	4.4366	84.49	0.0698	0.000655
7	3.6921	40.73	4.4203	83.12	0.0731	0.000553
8	3.5368	34.36	4.4182	82.94	0.0741	0.000448
9	3.4076	30.19	4.4339	84.26	0.0737	0.000346
10	3.2998	27.11	4.4445	85.15	0.0717	0.000251
11	3.2100	24.78	4.4576	86.28	0.0747	0.000166
12	3.1444	23.21	4.4693	87.30	0.0718	0.000096
13	3.0993	22.18	4.4762	87.90	0.0749	0.000044

Final Results:

Metric	Training	Validation	Test
Loss	3.0993	4.4182	4.4294
Perplexity	22.18	82.94	83.88
BLEU Score	-----	0.0749	0.0709
Best Epoch	-----	Epoch 8	-----

Observations:

- Higher LR caused 3x faster convergence -Exp2 matched Exp1's final PPL (107) by epoch 3 alone
- Overfitting began at epoch 8: val loss started rising while train loss kept falling
- Early stopping triggered at epoch 13 (patience=5 exhausted after epoch 8)
- Best model saved from epoch 8 (val_loss=4.4182, val_PPL=82.94)
- CosineAnnealingLR reduced the high LR gradually, preventing instability in early epochs
- Large train/val gap (Train PPL=22 vs Val PPL=83) by epoch 13 confirms overfitting



7. Experiment Comparison & Analysis:

Results Comparison Table

Metric	Exp 1 (lr=3e-4)	Exp 2 (lr=1e-3)
Test Loss	4.6760	4.4294
Test Perplexity	107.34	83.88
Test BLEU Score	0.0618	0.0709
Best Val PPL	107.11	82.94
Best Val BLEU	0.0624	0.0749
Epochs to PPL < 200	3	1
Epochs to PPL < 120	6	3
Total Epochs Trained	15	13
Early Stopping	No	Yes (ep.13)

Effect of Learning Rate — Analysis:

- **Positive Effect: Faster Convergence:** The model wasted no time. Larger weight updates meant it escaped poor starting points quickly and made meaningful progress in far fewer steps. What took 15 epochs before was now achieved in just 3 — a clean 5x speedup in convergence. For anyone watching the training curves, it felt like the model suddenly woke up.
- **Negative Effect: Overfitting:** But speed came with a cost. After epoch 8, something familiar and frustrating appeared — validation loss started climbing while training loss kept falling. The model had stopped learning general patterns and started memorizing specific ones. It knew the training data by heart — and was slowly becoming useless on anything new. Thankfully, early stopping caught it in time, preserving the epoch 8 checkpoint before the damage became irreversible.
- **Why CosineAnnealingLR Helped:** The solution wasn't to abandon the high learning rate — it was to tame it over time. The CosineAnnealingLR scheduler started aggressive and gradually eased the learning rate all the way down toward $1e-6$ — smoothly, naturally, without any jarring transitions. The result was the best of both worlds. Fast, energetic learning early on. Careful, stable convergence later. It's a combination so reliable it has become standard practice in Transformer training — and this experiment showed exactly why.

8. Conclusion:

Starting from an empty file and a blank canvas, a complete **decoder-only Transformer language model** — affectionately called **MiniGPT** — was built entirely from scratch in PyTorch. Nearly **11 million parameters**, trained on real Stack Overflow Python Q&A data, with a full pipeline running end-to-end — from raw messy data all the way to interactive inference.

The Two Experiments — A Tale of Two Learning Rates:

Two experiments were run, differing in just one thing — the learning rate.

- **Experiment 1 ($lr=3e-4$)** played it safe. Training was stable and steady, but slow — taking the full 15 epochs to converge. Final perplexity landed at **107.34** with a BLEU score of **0.0618**

- **Experiment 2 (lr=1e-3)** pushed harder — and it paid off. The model converged faster, learned better, and finished stronger — hitting a perplexity of **83.88** and a BLEU score of **0.0709** in just 13 epochs. That's **22% lower perplexity** and **15% higher BLEU** — a meaningful gap from one small change.
- The winning formula was clear — **high initial learning rate + cosine decay + early stopping**. Fast early learning, smooth late convergence, and a safety net that caught overfitting before it caused real damage.

The Core Insight:

Learning rate isn't just a number you set and forget. It's one of the most consequential decisions in Transformer training. Too low and the model crawls toward a solution it didn't need to struggle for. Too high and it sprints confidently in the wrong direction. The sweet spot — a high start with a graceful decay — gave this model its best possible chance.

Honest Limitations — What Held It Back:

No project is complete without honesty about its boundaries — and this one has a few worth acknowledging.

Only **3,000 samples** were used from a dataset containing over **600,000** — barely scratching the surface of what was available. Training on CPU forced the model to stay small — just **128 embedding dimensions** and **3 layers** — far below what a proper GPU training run could support. Sequences were capped at **128 tokens**, meaning longer answers were quietly cut off mid-thought.

The generated answers, while promising in structure, are still partially incoherent — and BLEU scores hovering around **0.06-0.07** reflect that honestly. Acceptable for a small model built from scratch. Far from production quality. But none of that diminishes what this project truly is — a complete, working, genuinely understood language model built by hand. Not borrowed, not fine-tuned, not wrapped around someone else's work. Built. From scratch. End to end.

Links: <https://chatgpt.com/share/69fa1c00-9448-8320-b335-bf7ed95bcea6>